

Section A:

You will be intervened on the following case study (you will need to understand all the code given):

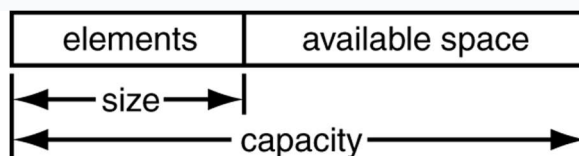
http://www.cs.sfu.ca/CourseCentral/125/tjd/vector_example.html

Section B

1. Write your own vector class `MyIntVector` (You will have 2 files `MyIntVector.h` contains the declaration and `MyIntVector.cpp` contains the implementation) to use as a “smart array” with integers. Do not use the build in vector `<vector>`.

Your `MyIntVector` must only have the following 3 private attributes: (more and you get 0 marks)

- `m_size`: An Integer, the number of items currently in the vector, which is directly controlled by how many elements you put into the vector.
- `m_capacity`: An integer, Capacity is the amount of space that the vector is currently using. Under the hood, a vector just uses an array. The capacity of the vector is the size of that array. This is always equal to or larger than the size. The difference between them is the number of elements that you can add to the vector before the array under the hood needs to be reallocated (a new, larger array block of memory being allocated, and the existing items being copied to it)



- `arrPtr`: A pointer to the first element of the underlying array

The only public methods your `MyIntVector` has are given below: (more and you get 0 marks)

- `push_back(int val)` to add an element to the end/back of the vector
- `size()` to return the no of elements in the array
- `capacity()` to return the amount of space currently allocated for the array
- `resize(int i)` to change the size of the array – setting new elements to zero if `i` is larger than the present size of the array, or chopping off the end of the array if smaller
- a default constructor `MyIntVector()` to initialise vector to a capacity of 10
- a destructor `~MyIntVector()` to free the memory allocated in the constructor
- two methods `at(int i)` and `operator[]` that return the element at index position `i` in your vector. These methods will be overloaded (see note below).

Note: the array index `operator[]`, does not do array bound checking, but the `at(int i)` method does do array bounds checking and if out of bounds, we will print an error message.

For robustness, these methods should return a const reference to the value (in this case a reference to an int `int&`) and are marked as a const methods, for example:

```
const int& at(int i) const;
```

Also, in order to facilitate their use as in `v.at(i) = value`, and `v[i] = value` both the `at()` and `operator[]` methods need to provide a second version which is not const, for example:

```
int& at(int i); // so we can say v.at(i) = value)
```

Section C

Write a main program ([source.cpp](#)) to test the [MyIntVector](#) class you have made in section B. Create objects of MyIntVector and test all your methods.

So you would have a class MyIntVector declared in a MyIntVector.h file, implemented in a MyIntVector.cpp file, and a source.cpp file, at the top of this file `#include "MyIntVector.h"` Then you can use the class in your source/main file to test it. For example:

```
#include "myintvector.h"
int main()
{
    MyIntvector v; //default constructor will set size to 10
    v.pushback(2); // push the integer 2 into the vector
    // more tests etc.
}
```